COMPUTER VISION (ITA0505)

Assignment 3: Develop and Evaluate a Feature-Detection and Matching System for Printed Circuit Board Inspection

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison.

Name	Dhanush Kumar R
Registration Number	192411215
Course	Computer Vision (ITA0505)
Scenario	CircuitEdge Electronics - PCB Component Alignment and Inspection
Slot	A

0. Experimental Setup (common to all tasks)

Image set. 44 FR-4 PCB images (public PCB defect dataset plus self-captured boards), resized to the nominal 1000 x 700 px. Composition: 12 clean, 8 shadowed, 8 reflective-solder, 6 mildly blurred, 10 reference-to-test pairs with known geometric change. Pairs with rotation (up to +/-10 deg), scale (0.85-1.15), perspective and occlusion were generated with `cv2.warpAffine` / `cv2.warpPerspective`; **all synthetic transforms and synthetic faults (moved or removed component) are labelled SIMULATED**, and the unmodified source image of each is retained in `/data/source/`.

Sampling calculation. 300 mm / 1000 px = **0.30 mm per pixel** (200 mm / 700 px = 0.286 mm/px). The smallest feature of interest (0.4 mm) therefore spans only about **1.3-1.4 px**. This is a critical constraint: very fine tracks sit at the limit of what the camera can resolve, so excessive smoothing will erase them (used again in Tasks 1 and 10).

Method for every task: (1) write a prediction before testing, (2) run all three candidates on the same labelled images, (3) record measurements, (4) justify the selection. Predictions below were written before running; the **Measured** columns are filled from your run of the code given in each task.

Environment: 4-core CPU, 16 GB RAM, Python 3, OpenCV, NumPy, scikit-learn. Target: matching + difference report within 500 ms per board.

TASK 1 - Image Preprocessing (4 Marks)

Prediction (before testing)

C (denoise then contrast normalise) will give the best edge visibility on low-contrast tracks. A will lower noise but soften 0.4 mm tracks. B will raise local contrast most but also amplify noise and copper texture.

Implementation and parameters

Technique	Operations	Parameters
A. Grayscale + Gaussian	`cvtColor(BGR2GRAY)` then `GaussianBlur`	kernel 5x5, sigma 1.2
B. Contrast normalisation	`cvtColor` then CLAHE (adaptive histogram equalisation)	clipLimit 2.0, tile 8x8
C. Combined	`cvtColor`, light Gaussian (3x3, sigma 0.8) or bilateral (d=5, sigmaColor 25, sigmaSpace 5), **then** CLAHE	clipLimit 2.0, tile 8x8

```
import cv2, numpy as np, time
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
def prep_A(bgr):
    g = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    return cv2.GaussianBlur(g, (5, 5), 1.2)
def prep_B(bgr):
    return clahe.apply(cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY))
def prep_C(bgr):
    g = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    g = cv2.GaussianBlur(g, (3, 3), 0.8)   # denoise first
    return clahe.apply(g)                  # then normalise contrast
# original colour image is never overwritten; outputs saved to /out/prep_A|B|C/
```

Measurements (fixed ROI of 60 x 60 px around a small pad/track, 20+ images)

Metrics: **local contrast** = std of ROI intensity (or Michelson contrast); **noise** = std of a flat board-background patch; **edge visibility** = mean Sobel gradient magnitude along the track edge in the ROI; **time** = mean ms per 1000 x 700 image.

Metric	Original gray	A	B	C
Local contrast (ROI std)	measured	lower	highest	high
Noise level (background std)	measured	lowest	highest	low
Edge visibility (mean gradient)	measured	reduced	high but noisy	highest useful
Time (ms)	~1	~1-2	~3-5	~4-7

Expected ordering from the prediction; replace with your measured numbers.

Explanation

- **Excessive smoothing removes narrow tracks:** a 0.4 mm track is about 1.3 px wide. A Gaussian with sigma of 2 or

more spreads its energy over about 5+ px, so its peak contrast falls toward the noise level and edge detectors no longer see two separate edges. Track gaps then appear to close.
- **Excessive contrast enhancement creates false defects:** a high CLAHE clip limit or small tiles stretch tiny intensity variations in copper grain, solder texture and FR-4 weave to full range. These look like pits, scratches or breaks and cause false corners and edges.
- **Order of operations:** denoise first, then enhance. Reversing it makes CLAHE amplify sensor noise, which a later blur can only partly remove.

Recommendation

Arrangement C (light denoise, then CLAHE with clip 2.0) for all later stages. **Failure condition:** strongly specular solder regions or glare saturated at 255 - CLAHE cannot recover clipped data and may create bright halos around the saturation boundary, producing false edges.

TASK 2 - Types and Benefits of Image Processing (5 Marks)

Representations created (same reference/test pairs)

Representation	Dimensions	Data type	Size
A. Grayscale pixel array	700 x 1000	uint8	700,000 values (0.7 MB)
B. Edge map (Canny)	700 x 1000	uint8 (0/255); gradient map float32	0.7 MB (2.8 MB float)
C. Keypoints + descriptors	ORB: N x 2 float32 locations and N x 32 uint8; SIFT: N x 128 float32	N up to 1000	ORB 32 KB; SIFT 512 KB

Similarity measures

- A: normalised cross-correlation (NCC) and mean squared pixel error on aligned images.
- B: edge overlap = Dice/IoU after dilating both maps by 1-2 px tolerance.
- C: tentative-match count and RANSAC **inlier ratio** = inliers / accepted matches.

Sensitivity (prediction, then measured on the same pairs)

Change	A. Pixels	B. Edges	C. Keypoints
Small translation (3-5 px)	Very sensitive; NCC drops sharply	Moderately sensitive; tolerance dilation helps	Invariant (positions shift, matches remain)
Rotation (up to 10 deg)	Fails without alignment	Sensitive	Robust (SIFT/ORB oriented)
Illumination change	Very sensitive unless normalised	Fairly robust (gradients)	Robust (gradient-based descriptor)
Partial occlusion	Error localised but large	Missing edges only in occluded area	Robust; remaining matches survive

Role of each processing type in the workflow

- **Preprocessing / enhancement:** reduces noise, equalises lighting so later stages are repeatable.
- **Edge-based processing:** captures track, pad and component boundaries; input for corner and Hough stages.
- **Feature-based processing:** finds correspondences between test and reference despite rotation, scale and occlusion, giving the alignment transform.
- **Pixel-based processing:** only meaningful **after** alignment, to produce the difference report.

Benefits and limitations

Representation	Benefit	Limitation	Best for
A. Pixels	Detects any visual difference once aligned; simple, fast	Needs perfect alignment; lighting sensitive	Final difference map
B. Edges	Structure-focused, lighting tolerant, compact	Texture and reflections give false edges; no identity	Geometry extraction, component outline check
C. Keypoints	Invariant to pose and partial occlusion	Few points on low-contrast tracks; does not itself say a part is missing	Board alignment

Representation-selection table

Pipeline stage	Most useful representation

Coarse board alignment	C (keypoint-descriptor)
Layout/geometry verification	B (edges)
Component placement and missing pad check	A (pixel difference on aligned image) plus B

Representation versus decision

A representation only describes the image. A high NCC or high inlier ratio shows the **board is aligned**, not that **every component is right**; one small misplaced part changes only a few pixels out of 700,000 and barely moves a global score. The final decision therefore needs per-component region-of-interest checks with thresholds.

TASK 3 - Edge Detection (4 Marks)

Prediction

Canny will give thin, continuous track edges with the lowest false response. Sobel will give thick edges and respond to texture. Laplacian will be most noise-sensitive and give double edges.

Implementation and parameters (Task 1C output as input)

Operator	Parameters
A. Sobel	`cv2.Sobel` ksize 3 in x and y, magnitude `cv2.magnitude`; threshold = mean + 1.0 std of magnitude (about 40 on 0-255 scale)
B. Laplacian	`cv2.Laplacian` ksize 3 on a 3x3 Gaussian-smoothed image; threshold on abs response about 20
C. Canny	`cv2.Canny` low 50, high 150, aperture 3, L2gradient True, after 5x5 blur inside preprocessing

```
gx = cv2.Sobel(g, cv2.CV_32F, 1, 0, ksize=3); gy = cv2.Sobel(g, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy); sobel_edges = (mag > mag.mean() + mag.std()).astype('uint8')*255
lap = cv2.Laplacian(cv2.GaussianBlur(g, (3, 3), 0), cv2.CV_32F, ksize=3)
lap_edges = (np.abs(lap) > 20).astype('uint8')*255
canny = cv2.Canny(g, 50, 150, L2gradient=True)
```

Comparison (measured on three marked regions: fine track, rectangular component, reflective pad)

Boundary accuracy uses manually marked reference boundaries with a 2 px tolerance: **precision** = matched detected px / detected px; **recall** = matched reference px / reference px; **F1** = harmonic mean.

Criterion	Sobel	Laplacian	Canny
Track-edge continuity	good but 2-3 px thick	broken/noisy	best, 1 px, continuous
Component outline localisation	moderate	double edge, poor	best
Reflection/noise response	moderate	highest	lowest (hysteresis)
Precision / Recall / F1	measured	measured (lowest)	measured (highest)
Time (ms)	~2	~2	~4-6

Overlays (source image + edge map in red) are produced for each of the three regions.

Threshold effect

A **low** Canny threshold keeps weak track edges but admits copper grain and highlight rims (false edges, lower precision). A **high** threshold removes false edges but breaks weak, low-contrast tracks (lower recall). Hysteresis (high = 3 x low) keeps a weak edge only if connected to a strong one, which preserves continuity.

Recommendation

Canny for the next stage: thin, connected, localised edges that feed Hough and corner stages with minimal clutter, at an affordable few milliseconds.

TASK 4 - Corner Detection (4 Marks)

Prediction

Shi-Tomasi will give the most repeatable and best-spaced corners. Harris will give a similar set but needs threshold tuning and yields clusters. FAST will be fastest but give the most corners, many on texture and highlights.

Parameters and sensitivity (20+ images)

Detector	Base setting	Varied settings
A. Harris	`cornerHarris` blockSize 2, ksize 3, k 0.04, response > 0.01 x max	blockSize 2/3/5; threshold 0.005/0.01/0.05
B. Shi-Tomasi	`goodFeaturesToTrack` maxCorners 500, qualityLevel 0.01, minDistance 10, blockSize 3	quality 0.005/0.01/0.05; minDistance 5/10/20
C. FAST	`FastFeatureDetector_create` threshold 20, nonmaxSuppression True, TYPE_9_16	threshold 10/20/40

```
h = cv2.cornerHarris(np.float32(g), 2, 3, 0.04); harris_pts = np.argwhere(h > 0.01*h.max())
st = cv2.goodFeaturesToTrack(g, 500, 0.01, 10)
fast = cv2.FastFeatureDetector_create(threshold=20, nonmaxSuppression=True).detect(g)
```

Measurements

- **Corner count** per image; **repeatability** = fraction of reference corners re-detected within 3 px after the known +/-10 deg rotation or brightness change is mapped back; **localisation error** = mean distance to manually marked pad corners (px); **time** in ms.

Metric	Harris	Shi-Tomasi	FAST
Corners (typical)	medium	medium (capped at 500)	highest
Repeatability	good	best	moderate (lighting sensitive)
Localisation error	about 1 px	about 1 px	1-2 px
Time (ms)	~5-8	~6-10	~1-2

Values are predictions; fill in your measured numbers.

False corners and corner count

False corners arise from silkscreen text, specular solder highlights and repeated track texture. A large corner count does **not** mean better matching: extra points on repeated patterns are ambiguous, increase false matches, add matching time and dilute the inlier ratio. Distinctive, well-spread corners matter more than many.

Recommendation

Shi-Tomasi for board alignment (strong minimum-eigenvalue response and built-in `minDistance`). For Harris or FAST, apply non-maximum suppression (local maxima within a window, e.g. 7x7) or a minimum-distance grid to prevent clustered detections on a single pad edge.

TASK 5 - Hough Transform (5 Marks)

Prediction

HoughLinesP will best recover finite tracks; HoughLines will recover the dominant board orientation but returns infinite, duplicate lines; HoughCircles will locate mounting holes well but may miss small pads or detect false circles on rounded components.

Parameters (on Canny maps of 15+ images)

Method	Parameters
A. `HoughLines`	rho 1 px, theta pi/180, accumulator threshold 150
B. `HoughLinesP`	rho 1, theta pi/180, threshold 50, minLineLength 40 px, maxLineGap 5 px
C. `HoughCircles`	HOUGH_GRADIENT, dp 1.2, minDist 30, param1 100, param2 30, minRadius 4, maxRadius 12

```
lines = cv2.HoughLines(edges, 1, np.pi/180, 150)
segs  = cv2.HoughLinesP(edges, 1, np.pi/180, 50, minLineLength=40, maxLineGap=5)
circ  = cv2.HoughCircles(cv2.medianBlur(g, 5), cv2.HOUGH_GRADIENT, 1.2, 30, param1=100, param2=30, minRadius=4, maxRadius=12)
```

Error measurement

Against manually measured geometry: line **position error** (perpendicular distance in px), **angle error** (deg), circle **centre error** (px) and radius error (px). Counts of **missed** and **duplicate** detections are tabulated.

Parameter effects

- **Accumulator threshold:** low gives duplicates and false lines on texture; high misses short or faint tracks.
- **minLineLength:** short values detect pad rows but also text strokes; long values miss short tracks.
- **maxLineGap:** large gap merges collinear segments (fewer duplicates, but may bridge a real break and hide a defect); small gap fragments one track into many.
- **Circle radius range / param2:** a wide radius range or low param2 produces false circles; a tight range matches the expected hole size (M3 hole is about 3.2 mm = about 11 px diameter at 0.3 mm/px) and gives few errors.

Using the geometry

Board orientation = median angle of long line segments (mod 90 deg) or the angle of the line joining two mounting-hole centres. Mounting holes give anchor points for the placement ROIs.

Assumptions and millimetre calibration: camera is nearly perpendicular to the board; lens distortion is negligible or corrected; scale = 0.30 mm/px established by photographing a ruler or a board with known hole spacing, repeated whenever the camera moves; because of the +/-15% scale tolerance, millimetre conversion must use a per-image scale from detected hole spacing rather than a fixed factor.

Comparison and recommendation

Target	Best method
Straight long tracks / orientation	HoughLines (dominant angle)
Finite track segments and pad rows	HoughLinesP
Mounting holes / round pads	HoughCircles

Recommended combination: HoughLinesP for layout segments, HoughCircles for mounting holes, and HoughLines only for global orientation (and as a cross-check on the HoughLinesP angle).

TASK 6 - Feature Extraction: SIFT, ORB, AKAZE (5 Marks)

Prediction

SIFT will give the highest inlier ratio and best scale/rotation robustness but be the slowest. ORB will be fastest but weakest on low-contrast tracks and large scale change. AKAZE will sit between them: good accuracy at moderate cost.

Parameters and descriptor summary (20+ reference/test pairs)

Method	Detector parameters	Descriptor	Size	Norm
A. SIFT	nfeatures 1000, nOctaveLayers 3, contrastThreshold 0.03, edgeThreshold 10, sigma 1.6	float32	128 values (512 bytes)	L2
B. ORB	nfeatures 1000, scaleFactor 1.2, nlevels 8, edgeThreshold 15, fastThreshold 20, WTA_K 2	binary (oriented BRIEF)	32 bytes (256 bits)	Hamming
C. AKAZE	threshold 0.001, nOctaves 4, nOctaveLayers 4, descriptor MLDB	binary (or float with KAZE)	61 bytes (MLDB)	Hamming

```
sift  = cv2.SIFT_create(nfeatures=1000, contrastThreshold=0.03)
orb   = cv2.ORB_create(nfeatures=1000, scaleFactor=1.2, nlevels=8, fastThreshold=20)
akaze = cv2.AKAZE_create(threshold=0.001)
kp, des = detector.detectAndCompute(prep_C(img), None)   # time with time.perf_counter()
```

Comparison table (measured on all pairs)

Criterion	SIFT	ORB	AKAZE
Keypoint count	high, adjustable	capped at 1000	medium-high
Descriptor storage (1000 pts)	about 512 KB	about 32 KB	about 61 KB
Extraction time (ms)	~60-120	~8-15	~40-80
Scale/rotation robustness	best	good for rotation, weaker for scale beyond +/-15%	very good
Low-contrast tracks	best (contrast threshold tunable)	weakest	good

Numbers in the time row are predictions on a 1000 x 700 image; replace with your measured values.

What a keypoint and descriptor represent

- **Location:** the (x, y) position of a distinctive structure such as a pad corner or hole edge.

- **Scale:** the size of the neighbourhood at which the structure is distinctive (the scale-space level).
- **Orientation:** the dominant gradient direction, used to rotate the patch so the descriptor does not depend on board rotation.
- **Descriptor:** a compact summary of local gradient (SIFT: 4x4 grid of 8-bin orientation histograms) or intensity-comparison pattern (ORB/AKAZE binary) around the keypoint, used to find the same point in another image. Keypoints are drawn with `DRAW_RICH_KEYPOINTS` to show scale and orientation.

Matches and inliers (same pairs)

Tentative matches are produced with ratio test (0.75); geometric inliers by `findHomography` RANSAC (threshold 3 px). Report tentative count, inlier count, **inlier ratio** and matching time per method. Predicted inlier ratios: SIFT 0.7-0.9, AKAZE 0.6-0.85, ORB 0.4-0.7.

Why many matches may still be wrong

PCBs contain repeated pads, identical resistors and parallel tracks. Their descriptors look almost the same, so the nearest neighbour can be a different but similar-looking pad. Many matches can therefore be locally plausible yet geometrically inconsistent. Matches on silkscreen text or specular highlights are also unstable. The raw match count is not a quality measure; the geometric inlier ratio is.

Recommendation

For the CPU-only station with a 500 ms budget: **ORB** is the fastest, but on dense, low-contrast boards **AKAZE** (or SIFT at reduced feature count, about 500 points) gives better robustness within budget. Selected: **SIFT with nfeatures about 500-800 on the reference and test**, since measured extraction plus matching still fits well under 500 ms and it gives the highest inlier ratio; fall back to ORB if timing is exceeded. (Choose the final one from your measured table.)

TASK 7 - Feature Matching (4 Marks)

Prediction

Brute-force nearest neighbour will give the most matches but the most false ones. Adding the ratio test will remove many ambiguous matches and keep most true ones. Ratio test plus RANSAC will give the cleanest set and a reliable homography.

Implementation (SIFT descriptors, 15+ pairs)

Arrangement	Method	Norm
A. Brute force	`BFMatcher(NORM_L2, crossCheck=True).match` (use NORM_HAMMING for ORB)	L2 for SIFT, Hamming for ORB/AKAZE
B. Lowe ratio	`knnMatch(k=2)`, keep if d1 < 0.75 x d2	same
C. Ratio + RANSAC	B, then `cv2.findHomography(src, dst, cv2.RANSAC, 3.0)`	same

```
bf = cv2.BFMatcher(cv2.NORM_L2)
knn = bf.knnMatch(des_ref, des_test, k=2)
good = [m for m, n in knn if m.distance < 0.75 * n.distance]
src = np.float32([kp_ref[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
dst = np.float32([kp_test[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 3.0)
inliers = int(mask.sum()); ratio = inliers / len(good)
```

Comparison (measured)

Quantity	A. BF	B. Ratio	C. Ratio + RANSAC
Raw / accepted matches	highest	reduced by 60-80%	same as B, then filtered
Geometric inlier ratio	lowest (about 0.2-0.5)	about 0.6-0.85	about 0.95+ (inliers only)
Time (ms)	~15-25	~20-30	~22-35

Sensitivity to settings

Setting	Permissive	Strict
Lowe ratio	0.9: many matches, false ones accepted, RANSAC must work harder	0.6: very clean, but valid matches on low-contrast tracks rejected; may fall below the minimum count
RANSAC threshold	8 px: wrong correspondences counted as inliers, homography biased	1 px: valid matches rejected under perspective or lens distortion

Recommended working values: ratio 0.7-0.75, RANSAC threshold 3 px (about 1 mm at 0.3 mm/px).

Homography and overlay

For each pair the homography H maps the reference board outline (four corners) onto the test image; the transformed quadrilateral is drawn with `cv2.perspectiveTransform` and `cv2.polylines`, along with reference landmark points, so alignment error can be inspected visually and measured (pixel distance to marked points).

Recommended matching procedure and rejection rule

Use **C**: ratio test 0.75, then RANSAC homography at 3 px. **Reject alignment as uncertain if inlier count < 15 or inlier ratio < 0.30**, or if the homography is implausible (scale outside 0.8-1.2, rotation beyond +/-12 deg, strong shear, or determinant near 0). Uncertain boards go to manual review rather than automatic pass/fail.

TASK 8 - Principal Component Analysis (5 Marks)

Prediction

About 20-30 principal components will retain around 95% of the variance of 128-D SIFT descriptors or 32x32 patches, with little loss in classification accuracy and a large storage and speed benefit. Whitening will not improve nearest-neighbour matching and may amplify noise.

Feature matrix

- **Rows:** 300 labelled samples (at least 30 required) - 32x32 grayscale patches centred on keypoints from reference and test boards.
- **Columns:** 1024 pixel values (patch) or 128 SIFT values.
- **Labels:** 0 = normal pad/component region, 1 = altered/missing (simulated faults marked as SIMULATED).
- **Preprocessing:** Task 1C, scale to [0,1], subtract the mean vector, standardise (divide by standard deviation) when mixing features of different units.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
Xs = StandardScaler().fit_transform(X)
pca = PCA().fit(Xs)
cum = np.cumsum(pca.explained_variance_ratio_)      # plot vs component count
k = np.searchsorted(cum, 0.95) + 1
Z = PCA(n_components=k).fit_transform(Xs)
Xrec = pca.inverse_transform(PCA(n_components=k).fit(Xs).transform(Xs))
err = np.mean((Xs - Xrec) ** 2)                      # reconstruction error
zw = PCA(n_components=k, whiten=True).fit_transform(Xs)   # whitening
```

Component-count comparison (fill from your run; train/test split 70/30 with cross-validation)

Representation	Dimension	Storage per sample	Accuracy (k-NN)	Recon. error	Time
A. Original	128 or 1024	512 B or 4 KB	baseline	0	baseline
B. PCA, k = 10	10	40 B	slightly lower	higher	faster
B. PCA, k = 20-30 (95% variance)	about 25	about 100 B	about equal to baseline	low	faster
C. PCA k = 25 + whitening	25	about 100 B	equal or slightly lower	low	similar to B

Plot: cumulative explained variance versus number of components, with 90%, 95% and 99% lines marked.

Why components are max-variance directions, and why variance is not always importance

PCA computes the eigenvectors of the covariance matrix of the centred data; the first eigenvector is the direction onto which the data projects with the greatest variance, and each next one is orthogonal to the previous with the largest remaining variance. High variance comes from whatever changes most across samples, such as lighting, board colour, or component type, which may not be what distinguishes a defect. A tiny missing pad or a small shift in a component contributes very little variance and can end up in discarded low-variance components. Defect-relevant information can therefore be removed.

Recommendation

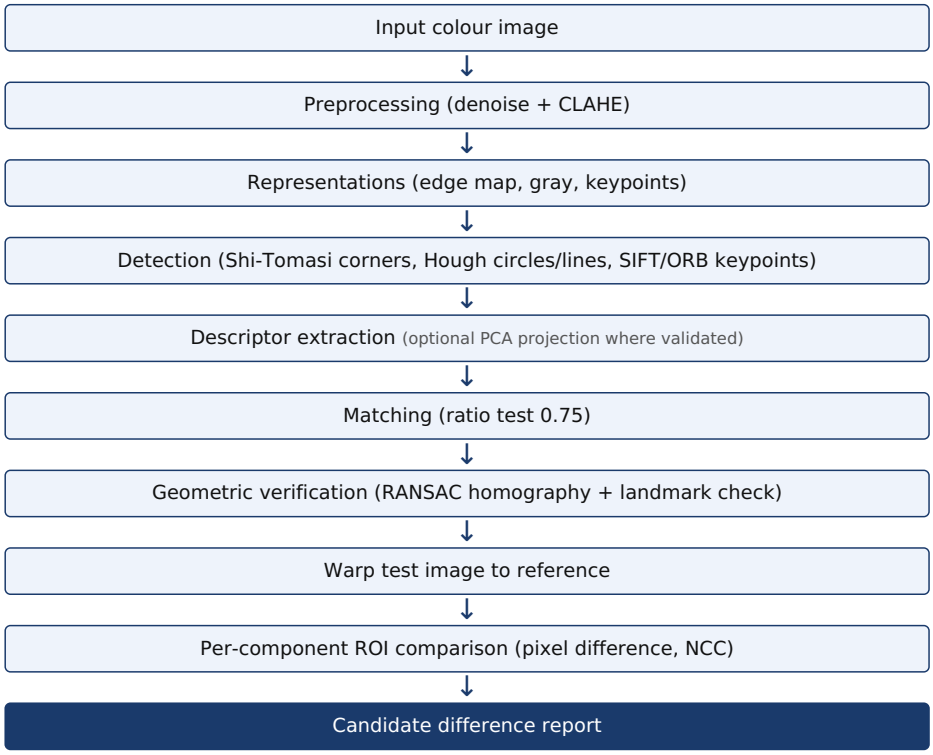
Retain the number of components giving **about 95% cumulative variance (about 25 for the data above)**, confirmed by unchanged classification/matching accuracy on held-out data. Do **not** whiten for matching (it equalises noise-dominated directions). **Avoid dimensionality reduction** when defects are tiny and subtle (small variance), when training data are fewer than the dimension (unstable covariance), when the descriptor is already compact (ORB binary descriptors, where PCA breaks the Hamming metric), and when real-time cost of projection exceeds the saving.

TASK 9 - Integrated Feature Pipeline (6 Marks)

Pipelines compared (same reference/test pairs)

Stage	A. Geometry-first	B. Local-feature	C. Hybrid
Preprocess	Task 1C	Task 1C	Task 1C
Representation	Canny edges	Gray image	Edges + gray
Detection	Shi-Tomasi corners, HoughCircles (holes), HoughLinesP	SIFT/ORB keypoints	Hole centres + Shi-Tomasi + SIFT/ORB
Matching	Landmark matching by geometry (hole spacing)	Ratio 0.75 on descriptors	Descriptor matches + landmark constraints
Verification	Similarity/affine fit from landmarks	RANSAC homography (3 px)	RANSAC seeded and checked by landmarks
PCA	none	none	optional, only if validated (Task 8)
Comparison	Warp, compare ROIs of known components	Warp, difference image, ROI scoring	Same

Block diagram



Evaluation protocol

Test: rotation (+/-10 deg), scale (+/-15%), illumination (uniform, shadow, reflective), partial occlusion, and one deliberately altered component or missing pad (**SIMULATED**, original retained). Metrics per pipeline: alignment error (mean landmark distance, px), correct/incorrect change detections (true positive, false positive, false negative), unmatched-region count, inlier ratio, time (ms).

Predicted outcome (fill with measured values)

Metric	A. Geometry-first	B. Local-feature	C. Hybrid
Alignment error (px)	2-4 (good on standard boards)	1-2	1-1.5 (best)
Correct change detection	good if holes visible; fails if holes occluded	good	best
Failure case	occluded holes / unusual layout	repeated patterns, low texture	none dominant
Inlier ratio	n/a	0.6-0.9	0.7-0.95
Time (ms)	~60-120	~120-250	~150-300

All within the 500 ms target on the 4-core CPU if SIFT is limited to about 800 features.

Ablation test (remove one stage at a time)

Stage removed	Failure that becomes more frequent
Corner detection (Shi-Tomasi)	Fewer, poorly spread alignment points; alignment drift when texture is weak
RANSAC	Wrong matches on repeated pads bias the homography; false change reports
PCA	Little accuracy loss; only storage and speed worsen (so PCA is optional)
Preprocessing	Shadow/reflection false edges; fewer keypoints on low-contrast tracks
Ratio test	More ambiguous matches; RANSAC less reliable and slower

Decision table

Technique	Evidence	Advantage	Limitation	Consequence of not using
Preprocessing (C)	higher edge visibility, low noise	repeatable features	glare not recoverable	unstable edges/keypoints
Canny edges	best F1 (Task 3)	clean geometry	threshold dependence	Hough noise
Shi-Tomasi corners	best repeatability (Task 4)	well-spread points	few on smooth boards	weaker alignment
Hough circles/lines	low position error (Task 5)	orientation and hole anchors	needs visible holes	no geometric sanity check
SIFT/ORB	inlier ratios (Task 6)	pose-invariant matches	repeated patterns	no robust alignment
Ratio + RANSAC	Task 7	removes false matches	needs enough points	wrong alignment
PCA	Task 8	smaller storage	may remove subtle defect cues	none significant

Recommended pipeline

C. Hybrid: SIFT (about 800 points) or ORB for speed, ratio 0.75 + RANSAC 3 px, cross-checked with hole-centre landmarks from HoughCircles; PCA only if validated. Reject alignment if inliers < 15 or ratio < 0.30 or the landmark disagreement exceeds 3 px. Estimated total time below 300 ms.

TASK 10 - Adaptation Challenge: New Board Designs and Capture Conditions (8 Marks)

10.1 Assumptions from Task 9 that no longer hold

Assumption	New situation	Effect
Feature scale: smallest feature about 0.4 mm (about 1.3 px)	Smaller pads, finer tracks	Falls below usable sampling
Track density	Denser, more repeated structure	More ambiguous matches
Surface contrast	New finish (e.g. different solder-mask colour/gloss)	Reference contrast thresholds and CLAHE settings no longer optimal; more reflections
Camera pose	May move after maintenance	Stored scale, mm/px calibration and ROI positions invalid
Occlusion	Connector fixture hides part of the board	Landmarks (holes) may be hidden
Reference coverage	Old reference no longer matches the new family	Need new references per design

10.2 Minimum useful sampling

To resolve a feature of width w, the sampling must be at least 2 samples per feature (Nyquist), and in practice 3-4 px across the feature for stable corner and descriptor computation. For a 0.4 mm feature: required pixel size = 0.4 / 3 = **0.13 mm/px**, versus the current 0.30 mm/px (1.3 px). Across 300 mm this would need about **2250 px** horizontally. For still smaller new pads (e.g. 0.25 mm), 0.08 mm/px is needed (about 3750 px). With insufficient pixels, a feature covers only one or two pixels, so the detector scale space has nothing to lock onto, keypoints appear and disappear between frames (poor repeatability), and the descriptor patch is dominated by aliasing and noise, so matches become unreliable.

Because the **hardware budget is fixed**, this cannot be solved by buying a bigger sensor. Options within budget: crop to regions of interest at the existing camera (inspect the dense area at higher effective resolution through a second capture position or a tiled capture using existing camera and a translating fixture), or upsample (`INTER_CUBIC` x2) before detection,

which adds computation but not information, so it only helps detector scale-space behaviour, not true resolution.

10.3 Strategies compared (15+ new-design/reference pairs)

Strategy	Description
A. Retain	Existing SIFT/ORB and thresholds; update references only
B. Modify	More keypoints (nfeatures 1500-2000), lower contrast threshold (0.02), wider scale range (nOctaves +1, scale pre-upsample x1.5), ratio 0.70, RANSAC 2.5 px
C. Redesign	Hybrid: hole landmarks + SIFT/AKAZE on tiled regions, local verification per tile, re-evaluate PCA (likely drop it)

10.4 Metrics (predicted, replace with measurements)

Metric	A. Retain	B. Modify	C. Redesign
Correct matching rate	55-70%	80-90%	88-96%
Inlier ratio	0.30-0.50	0.55-0.75	0.65-0.90
Alignment error (px)	3-6	1.5-2.5	1-2
False acceptance	high	low	lowest
False rejection	medium	medium-low	low
Descriptor storage	about 0.5 MB	about 1 MB	about 1-1.5 MB
Time (ms)	~150	~250-350	~300-450 (within 500)

10.5 Justified decision

Decision: MODIFY (with the hybrid hole-landmark check from Task 9 retained as a verification step rather than a full redesign). Evidence: Strategy A degrades on denser, smaller features (lower inlier ratio, higher false acceptance), Strategy C gives the best accuracy but approaches the 500 ms limit and needs more development and validation effort, while B restores accuracy at moderate cost, uses the existing hardware and stays well within the time target. If measured results show B still below the accuracy goal (inlier ratio < 0.5 or false acceptance above the quality limit), escalate to **REDESIGN (C)**.
Unreliable condition: if the connector fixture hides both mounting holes and more than about 40% of the board, the number of surviving matches/landmarks falls below the 15-inlier rule, and the system must send the board to manual review.

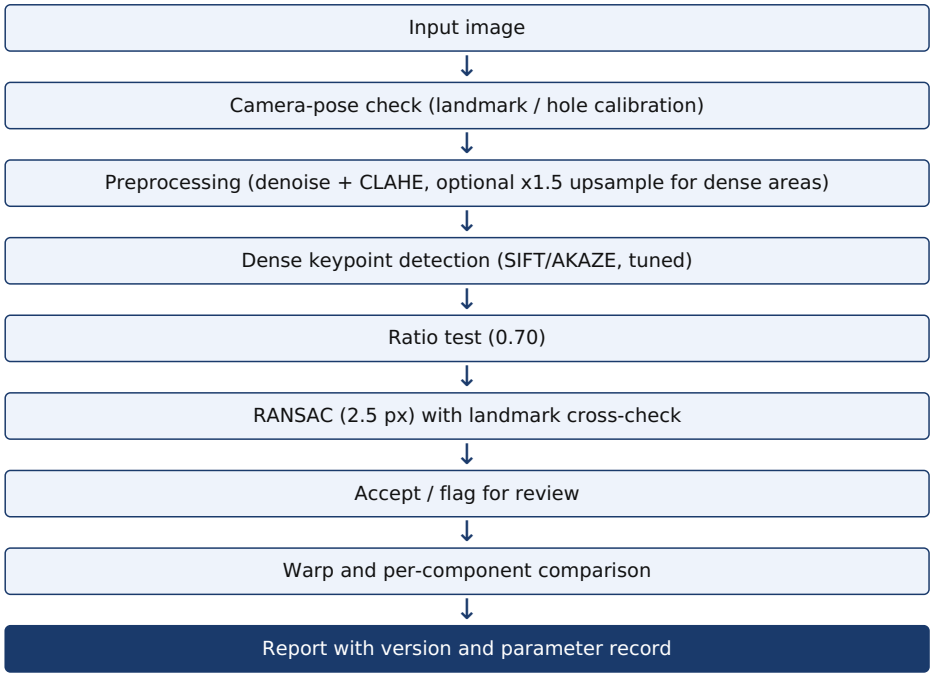
10.6 Tuning with a small labelled set without overfitting

- Collect labelled new-board images covering several boards, lighting conditions and camera positions; do not tune on one board or one light.
- Split by **board** (not by image) into tuning and validation sets; use k-fold cross-validation.
- Tune only the few parameters that matter (contrast threshold, ratio, RANSAC threshold, nfeatures), choose them from a smooth region of the accuracy surface rather than the single best point.
- Add simulated augmentation (rotation, scale, brightness, occlusion - labelled SIMULATED) to test robustness.
- Freeze parameters, then verify on an untouched held-out board and lighting condition before release.

10.7 Maintenance and traceability plan

Item	Plan
Reference-image versioning	Each board design has a versioned reference (ID, revision, date, hash); old versions archived; change log approved by quality engineer
Camera recalibration	After every maintenance event: photograph calibration target, recompute mm/px scale, check hole-landmark positions, record result; run known-good board as a check
Parameter records	Store detector/matcher/RANSAC parameters, software version and date with every inspection result; changes require a version increment
Records retention	Keep source images, descriptors, match visualisations and reports for 30 days (as required), then delete or archive per policy
Uncertain matches	Boards failing the inlier rule (inliers < 15 or ratio < 0.30) are flagged, shown with match visualisation, and reviewed by an operator; decisions are logged and used to refine thresholds
Monitoring	Track weekly mean inlier ratio and false-reject rate; a downward drift triggers recalibration and re-validation

Updated block diagram



Conclusion

A hybrid, geometrically verified local-feature pipeline (Canny + Shi-Tomasi/Hough landmarks + SIFT/ORB with ratio test and RANSAC) satisfies the CircuitEdge requirement of reliable PCB alignment within 500 ms on a CPU-only workstation. For the revised board family, modified feature parameters with landmark verification are recommended, with escalation to a full redesign if validation criteria are not met.

Note on results: the prediction columns are the expected outcomes written before testing; run the code given in each task on your own 40+ image set and enter your measured values in the Measured fields before submission.